

COMPSCI 340 2026 S2 — Rust Version

Assignment 2 (50 pts)

xv6 is a modern re-implementation of Unix designed as a teaching operating system. Its codebase is much smaller than a production operating system, making it easier to read, understand, and modify. By working with xv6, you can gain important insights into fundamental operating system concepts while dealing with less complexity than in typical OS kernels.

This assignment focuses on becoming familiar with how virtual memory is implemented in xv6. You will complete a series of three tasks focused on different aspects of kernel memory management.

You can assume that we will not be testing error cases, except where error handling is explicitly required in the task description.

1 Development Environment

1.1 Installation

This assignment needs the same Unix environment and base repository as the first assignment. You can refer to the first assignment if you need to set up your environment again.

This assignment uses the same base as the first assignment. If you still need to setup your environment, please follow the instructions there.

Please **do not** include modifications from previous assignments, as this may interfere with the marking. You should remove the folder in which you completed the previous assignment and then clone the repository using the following command:

```
git clone https://github.com/UoA-CS340-SE370-OSDev/xv6-rust-student
```

1.2 Versions of this Assignment

There is both a C and a Rust based version of this assignment available, you can choose to complete the assignment in either language, but **you must complete the entire assignment in one language**. This is the **Rust** version of the assignment. If you want to complete this assignment using C then you should switch to the other version as it contains the C specific instructions.

You may choose to complete future assignments in a different language than the one you use for this assignment.

2 Overview of Virtual Memory in xv6 for RISC-V

xv6 uses the RISC-V Sv39 virtual memory addressing scheme: 39-bit virtual addresses with 4KiB pages and a three level page table.

2.1 Page Table Entries

A page table entry is represented by the `pte_t` type which is an unsigned 64-bit integer type.

Bits 0-9 store the page table entry flags, and the rest of the bits store the physical frame address. The bits represent the following flags:

- Bit 0: Valid flag.
- Bit 1: Read permission flag.
- Bit 2: Write permission flag.
- Bit 3: Execute permission flag.
- Bit 4: User accessible flag.
- Bits 5 through 9 are not used in xv6.

Macros to work with page table entries can be found in `kernel/riscv.h`

2.2 Translation Lookaside Buffer (TLB)

The TLB is used by the CPU to cache pagetable lookups. If you need to modify a pagetable that is in use, then the function `sfence_vma()` needs to be used to clear the TLB, as otherwise the old cached page table entry could continue being used by the MMU.

3 Task 1: Displaying Used Memory (15 Points)

In this task, you will create a system call to query the total used memory. This task consists of two parts: implementing a system call to query the total allocated memory, and creating a user program to call the syscall from the command line.

3.1 Syscall `getusedmem`

For the syscall, you will need to traverse the multi-level kernel page table and count the number of valid pages. You will need to:

- Place the syscall function in `kernel/src/sysproc.rs`.

- Modify the `kernel/src/vm.rs` file to add a function that walks the kernel page table, `walk_used`.
- You will need to add a public getter function to the `Kvm` struct, `pub fn get(&self) -> &PageTable`.
- Have a look at the `free_walk` function in the `PageTable` struct.

Hints

3.2 User program

Create a user program `used` that provides the following output:

```
Used memory: [mem] bytes
```

Where `[mem]` is the currently allocated memory value (in bytes), retrieved from the system call (e.g. 50000).

This program should be found in `user/bin/used.rs`, and requires you to do the normal steps for adding a user program to `xv6`.

4 Task 2: Read Only Pages (20 Points)

In this task you will implement two system calls `mprotect` and `munprotect` to change the write permissions of a given page. In your implementation you will need to check if the given address is a valid page address, and return an appropriate error code if it is not.

Use the signatures

```
pub fn sys_mprotect(args: &SyscallArgs) -> Result<usize, SysError> and
pub fn sys_munprotect(args: &SyscallArgs) -> Result<usize, SysError> in
kernel/src/sysproc.rs.
```

Create a function in the `PageTable` implementation with the following signature:

```
pub fn set_page_writeable (&mut self, va: VA, writable: bool) -> Result<(),
VmError>
```

4.1 Testing

You can use the following program to test your implementation:

```
#![no_std]
#![no_main]
```

```

use user::*;

#[unsafe(no_mangle)]
fn main(args: Args) {
    let addr = match sbrk(100).expect("sbrk failed!");
    let page = addr as *mut u8;

    // write to make sure memory has been allocated and is writable (set to 255)
    unsafe { *page.add(0) = (0xFF) as u8 };
    mprotect(addr);
    unsafe {
        // *page.add(0) = (1 & 0xFF) as u8; // if you uncomment this line,
        // the program stops, as the kernel kills it for trying to write to
        // a protected page
        println!("{}", *page.add(0)); // should still be 255
    }

    munprotect(addr);
    unsafe {
        *page.add(0) = (2 & 0xFF) as u8;
        println!("{}", *page.add(0)); // should read 2
    }
}

```

Example output

```

$ ropagetest
255
2
$

```

4.2 Hints

- Have a look at `walk_addr` in `kernel/src/vm.rs` to understand how to get a page from an address.
- You will have to create methods for setting a page's writability from a memory address (remember that processes only see virtual memory!)
- You will have to extend the `PageTableEntry` implementation to include a function that changes the `PTE_W` bit in the entry. The recommended signature is this:
`pub fn set_w(&mut self, flag: bool).`
- Setting a page entry value will mean it has to be mutable, which means that you will want to use `args.proc()...` inside an `unsafe` block.
- You do not have to add any error handling for when the kernel kills a process for attempting to write a read-only page.

5 Task 3: Null Pointer De-referencing Error (15 Points)

A pointer to the memory address 0 (known as a null pointer) is used in many languages to denote "no object". De-referencing a null pointer almost always indicates an error in the program. Unix based operating systems typically intentionally do not allocate the page starting at address 0 so that de-referencing a null pointer results in a Segmentation Fault (the SIGSEGV signal).

The rust version already has protection for memory address 0, however it does not tell the user when something goes wrong

Try the following program:

```
#![no_std]
#![no_main]

use user::*;

#[unsafe(no_mangle)]
fn main(args: Args) {
    let ptr: *const i32 = core::ptr::null();

    unsafe {
        println!("{}", *ptr); // attempt to dereference a null pointer
    }
}
```

Your task is to make the error message that is printed to the user more readable in the following format:

```
$ nullptrtest
Segmentation Fault!
```

5.1 Hints

- You may find the [Rust RISC-V crate documentation](#) useful (specifically the scause page)
- Read through the `exec` function in `kernel/src/proc.rs` to understand how the page protection works

6 Marking

6.1 Task 1

Criteria	Marks
<code>used.rs</code> makes use of the <code>getusedmem</code> syscall	3
The <code>getusedmem</code> syscall makes use of the <code>walk_used</code> function	3
The <code>walk_used</code> function in <code>vm.rs</code> walks the page table tree	4
The program outputs an expected output	5
Total	15

6.2 Task 2

Criteria	Marks
Both system calls have been added with expected signatures	4
Follows expected signatures and in <code>kernel/src/vm.rs</code>	4
Both system calls correctly handle invalid arguments and return with an appropriate error code.	4
Correct behaviour and output of the test program	8
Total	20

6.3 Task 3

Criteria	Marks
Finds the correct method to extend	3
Uses correct RISC-V primitives (doesn't hard-code values)	3
Kills the process that dereferences the null pointer	4
Output when a null pointer is dereferenced is correct	5
Total	15

7 AI Usage

This assignment is Lane 2 (Uncontrolled). Students may use AI to assist them, and this use is not restricted.

8 Submitting Your Assignment

Run the command `sh create_submission.sh` to create a zip file named:

`CS340_A2_RUST_<UPI>.zip`

Rename the file with your UPI and submit this zip file on canvas.